

Getting Started

Where the course material lives, how to get it, how the labs work, and the two accounts you must set up carefully before you touch anything.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

COURSE

24AIM332 Introduction to Cloud Computing

REPOSITORY

github.com/Amrita-School-of-AI/cloud-24aim332

READ THIS

In week one, before the first lab

TIME NEEDED

About 40 minutes, once

OFFERING

Odd Semester 2026-27

Read this once, in week one

Everything for this course lives in one public GitHub repository. This guide gets you from nothing to a working setup.

If you are also taking **23AID304 High-Performance and Cloud Computing**, Parts 2 and 3 below are the same as in that course's guide. Do them once.

BY THE END OF THIS GUIDE YOU WILL HAVE

- A GitHub account with a working SSH key.
- The course repository, and your own fork.
- A cloud account with a **spending alert already set**.
- Docker and `kubect1` installed, and a local Kubernetes cluster you can throw away.

Part 1: What is in the repository

github.com/Amrita-School-of-AI/cloud-24aim332

Public. No permission needed.

```
cloud-24aim332/  
├─ handbook/      week plan, marks, policies, tools  
├─ slides/        the lecture decks, including the two case studies  
├─ labs/          the lab sheets  
├─ exercises/     auto-checked exercises  
└─ submissions/   where your answers go
```

DECK	COVERS
Cloud fundamentals	What cloud is, service and deployment models, the economics
Virtualization and containers	Hypervisors, namespaces and cgroups, images, layers
Kubernetes	The reconciliation model, Deployments, Services, probes
IaC and CI/CD	Beyond the syllabus. Declarative infrastructure, pipelines
Case study: Echno	A construction-intelligence platform, and how it is deployed
Case study: a digital twin on Azure	What production rigour costs, and what it catches

NOTE · THE TWO CASE STUDIES ARE REAL SYSTEMS

They are platforms I build and run, presented with the architecture **and the mistakes**: a template that would never have converged, a teardown ordering that stranded a certificate, a data service that answered a question it could not actually answer. Scrubbed of anything confidential; the engineering lessons are the point.

Part 2: A GitHub account and an SSH key

2.1 Create the account

Go to github.com and sign up. Use a username you would be happy for an employer to see. Add your Amrita email to claim the **GitHub Student Developer Pack**, which includes cloud credit that is useful for this course.

2.2 Make an SSH key

```
ssh-keygen -t ed25519 -C "your.email@example.com"
```

Press Enter three times for the defaults.

PITFALL · PRIVATE KEY VERSUS PUBLIC KEY

`~/.ssh/id_ed25519` is **private** and never leaves your machine. `~/.ssh/id_ed25519.pub` ends in `.pub` and is the one you share.

This distinction is not academic in this course. In week 4 you will go looking for exposed credentials in cloud configuration, and the same rule applies to every key and token you handle from here on.

2.3 Add it to GitHub

```
cat ~/.ssh/id_ed25519.pub
```

Copy the whole line. On GitHub: avatar → **Settings** → **SSH and GPG keys** → **New SSH key** → paste → **Add SSH key**.

2.4 Check

```
ssh -T git@github.com
```

`Hi <username>! You've successfully authenticated` is what success looks like, and the note about no shell access is expected.

If you get `Permission denied (publickey)`, you almost certainly pasted the private key or only part of the public one.

2.5 Identify yourself to git

```
git config --global user.name "Your Name"  
git config --global user.email "your.email@example.com"
```

Part 3: Get the material

Fork the repository (the **Fork** button, top right), then clone your fork:

```
git clone git@github.com:<your-username>/cloud-24aim332.git
cd cloud-24aim332
git remote add upstream https://github.com/Amrita-School-of-AI/cloud-24aim332.git
```

To pull new material during the semester:

```
git fetch upstream && git merge upstream/main
```

Part 4: Your cloud account, and the one thing to do first

You need **one** cloud account. AWS, Azure and Google Cloud all have a free tier sufficient for every lab here. Your Student Developer Pack may include credit.

PITFALL · SET A SPENDING ALERT BEFORE YOU CREATE A SINGLE RESOURCE

This is the most important paragraph in this guide.

The usual way a student on this course incurs a real bill is leaving a load balancer, a managed database or a GPU instance running over a holiday. Nothing stops you. Nobody warns you. The bill simply arrives.

Before anything else: set a **budget alert** at a small amount, and enable **multi-factor authentication** on the root or owner account. Then create a non-root user with only the permissions you need, and use that for all work.

Every lab in this course ends with a teardown step, and **teardown is marked**.

Part 5: Install the tools

Ubuntu, Debian, Linux Mint, or WSL2:

```
sudo apt update
sudo apt install -y git curl docker.io
sudo usermod -aG docker "$USER"      # then log out and back in
```

Kubernetes, locally:

```
# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -Ls
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# kind: a throwaway cluster that runs in Docker
curl -Lo ./kind https://kind.sigs.k8s.io/dl/latest/kind-linux-amd64
sudo install -o root -g root -m 0755 kind /usr/local/bin/kind
```

OpenTofu, for the infrastructure-as-code labs: follow the install instructions at opentofu.org .
Terraform works identically for everything in this course.

Check:

```
docker run --rm hello-world
kind create cluster --name icc && kubectl get nodes && kind delete cluster --name icc
tofu version
```

NOTE · A LOCAL CLUSTER COSTS NOTHING

`kind` runs a real Kubernetes cluster inside Docker on your laptop. Use it for every Kubernetes lab. Reach for a managed cluster only when you specifically want to see a cloud load balancer, and tear it down the same day.

macOS: install Docker Desktop, then `brew install kubectl kind opentofu` .

Windows: use WSL2 with Ubuntu and follow the Linux instructions inside it.

Part 6: How the marks work, and what the labs are for

COMPONENT	WEIGHT
Class assignment	50%
End - semester examination	30%
Class examination	20%

The class assignment is one system, not eight submissions

Fifty percent of this course is a single deliverable: a small application, **containerised, orchestrated, declared as infrastructure code, and delivered by a pipeline.**

STAGE	MARKS
Containerised application	10
Orchestrated on Kubernetes	12
Declared as infrastructure code	12
Delivered by a CI/CD pipeline	12
Report and teardown proof	4

The labs are how you build it

Eight labs, **not separately marked.** Each one produces one piece of the assignment:

LAB	YOU BUILD	FEEDS
1	First instance, budget alert, clean teardown	Teardown proof
2	Object storage from the CLI and from code	The application
3	A containerised application with a multi-stage build	Stage 1
4	A multi-container stack with persistent state	Stage 1
5	A Kubernetes deployment, service, probes, rolling update	Stage 2
6	Autoscaling under load	Stage 2
7	Infrastructure as code: provision, converge, destroy	Stage 3
8	A pipeline from commit to running deployment	Stage 4

PITFALL · DO THE LABS AS THEY COME

Nothing here is marked on its own, which makes it tempting to skip. But each stage depends on the one before, so students who keep up have the assignment nearly finished by week 14, and students who do not are learning four unfamiliar technologies at once, against a deadline, with a live cloud account and a real bill.

Keep your lab work in one repository as you go. That repository **is** the assignment submission.

PITFALL · PASTE TEXT, NOT SCREENSHOTS

Command output belongs in your report as **text**. It is searchable, readable at any size, and can be checked. A screenshot of a terminal is none of those. Screenshots are fine for a console UI where there is no text to paste.

Part 7: The exercises

Some exercises are checked automatically. They work like the labs but the artefact is a file, not a report.

```
cd exercises/icc-ex02-kubernetes-manifest
cp starter/manifest.yaml ../../submissions/icc-ex02-kubernetes-manifest/AIE23001.yaml
# edit it, then:
./selfcheck.sh ../../submissions/icc-ex02-kubernetes-manifest/AIE23001.yaml
```

The checker runs in stages and tells you exactly which requirement is unmet:

```
==> deployment
  ok Deployment present
  ok 3 replicas
FAIL: the selector {app: web} does not match the pod template labels {app: frontend};
      the Deployment would manage no pods
```

Submit one file, named for your roll number.

NOTE · SOME CHECKS NEED A RUNNING CLUSTER

`kubectl` contacts the API server even for a client-side dry run. With no cluster the checker skips schema validation and says so; the structural checks still run. Start `kind create cluster` first to get the full check.

Part 8: Rules

Late work. 80 percent within 48 hours; not accepted after that without documented cause.

Collaboration. Discuss freely. Submit only work you produced.

AI tools. Permitted for learning. You must be able to explain everything you submit, including every line of a manifest or a template. Cloud configuration produced without understanding is how expensive and insecure systems get built, so this matters more here than in most courses.

Secrets. Never commit a credential. A key pushed to a public repository is scanned and abused within minutes. A submission containing a live credential scores zero, and you rotate the key immediately.

Part 9: When something goes wrong

Bring: what you expected, what happened, the exact error as text, and what you have already tried. For cloud problems also bring the region, the resource type, and whether it ever worked.

SYMPTOM	CAUSE
<code>permission denied</code> from <code>docker</code>	You did not log out and back in after <code>usermod -aG docker</code>
<code>Permission denied (publickey)</code> on git	See Part 2.4
<code>kubectl</code> cannot connect	No cluster running. <code>kind create cluster</code>
A resource will not delete	Something still references it. Teardown has an order; see Lab 7
An unexpected charge	Check for load balancers, static IPs, managed databases and unattached disks. These survive deleting the instance

BEFORE YOUR FIRST LAB, CONFIRM

`ssh -T git@github.com` works · your fork is cloned · `create cluster` works
· a **budget alert** is set on your cloud account · MFA is on · `docker run --rm hello-world` works · `kind`

Welcome to the course.

Dr. Abhijith Anandakrishnan Assistant Professor, Amrita School of AI